Práctica 4: Técnica de Paso de Testigo

Programación de Sistemas Concurrentes y Distribuidos

Dpto. de Informática e Ingeniería de Sistemas,
Grado de Ingeniería Informática
Escuela de Ingeniería y Arquitectura
Universidad de Zaragoza

Objetivos

El objetivo de esta práctica es aplicar la técnica de *Paso de testigo*, traduciendo el diseño de la sesión anterior a una implementación basada en semáforos. En concreto, la práctica pretende:

- Comprender y aplicar la técnica de Paso de testigo,
- resolver problemas de sincronización de procesos utilizando semáforos,
- y profundizar en el modelo de concurrencia de C++.

Trabajo previo

Antes de la sesión en el laboratorio es imprescindible leer el enunciado, estudiar y comprender la técnica que se tiene que aplicar, y realizar un esquema básico del programa que se debe construir. También se recomienda revisar todo el material de apoyo publicado como parte de la sesión y que va a usarse en su desarrollo: clase `Semaphore_V4`, TAD `BounduedQueue`, y librerías y código para mostrar imágenes en una ventana.

Los contenidos teórico-prácticos necesarios para realizar esta práctica han sido presentados en la lección 6 de la asignatura, así como en las sesiones 8 y 9 de problemas.

Semaforos en C++

C++ no ha integrado los semáforos hasta su estándar 20. La versión que utilizamos, la 11, no los tiene. Por este motivo se suministra la clase `Semaphore_V4`, correspondiente a

los ficheros `Semaphore_V4.hpp` y `Semaphore_V4.cpp`. Se proporciona también un ejemplo de uso `pruebaSemaforos.cpp` para ver un ejemplo de manejo.

La clase implementada tiene un único constructor `Semaphore(const int n)`. El parámetro corresponde al número de permisos que se le asocia inicialmente, equivalente al usado en la notación algorítmica utilizada en clases de teoría.

La especificación de la clase semáforo ofrece dos versiones de `wait` y dos de `signal`. La diferencia entre ellas radica en el número de permisos que están involucrados en la acción. La versión sin parámetros (`sem.signal(), sem.wait()`) se corresponde con la explicada en clase (el valor del semáforo se incrementa/decrementa en una unidad respetando las reglas semánticas de la primitiva de sincronización). La semántica de la segunda versión (`sem.signal(3), sem.wait(2)`, por ejemplo), es una generalización de la anterior, que se explica en los comentarios del fichero `Semaphore_V4.hpp`.

Tareas a desarrollar

El punto de partida de esta práctica es el diseño resultante de la sesión anterior. Este diseño debe ser completo y finalista, no una versión aproximada, resolviendo con instrucciones de tipo <**await B S**> todas las cuestiones de sincronización involucradas en el problema: gestión de la concurrencia en el uso de las colas de peticiones y las estructuras de datos relacionadas con las respuestas a los clientes, y finalización controlada de los distintos procesos que intervienen en el sistema.

El objetivo del trabajo es traducir, aplicando la técnica de Paso de testigo, el diseño anterior a un programa en C++ donde las cuestiones de sincronización se resuelvan con semáforos. En concreto, los pasos a seguir son los siguiente:

1. Analizar el diseño y decidir cuántas exclusiones mutuas (o testigos) independientes se pueden programar para maximizar la concurrencia de la solución.

2. En base a la decisión anterior, aplicar la técnica de paso de testigo a cada exclusión mutua definida:
   a) Declaración de las variables necesarias para la exclusión mutua y sincronización de las instrucciones
   b) Aplicación de las plantillas de traducción de las instrucciones <**await B S**>, usando las variablñes anteriores
   c) Programación de la función (o funciones) *pasoTestigo(...)*

El resultado debe ser un programa en C++ que simule el funcionamiento del sistema de publicidad descrito en el enunciado de la práctica anterior.

Cuestiones importantes a tener en cuenta

A continuación se detallan brevemente una serie de cuestiones que se deben tener en cuenta a la hora de implementar la solución al problema:

1. El programa principal debe de contener, como mínimo: la declaración de los datos compartidos y la gestión del ciclo de vida de todos los procesos del sistema (clientes, publicadores y finalizador).

2. Las variables de sincronización resultantes de aplicar Paso de testigo deben declararse en el programa principal. Dado que estas variables se tiene que pasar como parámetros a los distintos procesos, si su número es muy elevado, se recomienda agruparlas y encapsularlas en un dato de tipo *struct*.

3. Cada proceso publicador debe ir parametrizado por la ventana (*ImageWindow*) que controla. Recordad que una vez que muestre una imagen concreta no podrá atender la siguiente petición como mínimo hasta que el tiempo de servicio especificado en la petición se haya completado.

4. Todas las imágenes deben estar dispnibles en una única carpeta para facilitar el acceso a las mismas por parte de los publicadores. En el material de apoyo para la realización de la práctica se incluye un directorio con 20 imágenes.